

Birds' Track: Tracking Objects from dynamic background with OpenCV

Anonymous CVPR submission

Paper ID *****

Abstract

Tracking fast-moving objects in unconstrained natural videos remains a challenging problem due to background dynamics, illumination changes, camera motion, and scale variation. Classical motion-based pipelines offer efficiency and interpretability, whereas modern deep-learning-based trackers achieve high performance at greater computational cost. This work revisits classical computer vision approaches for multi-object tracking in the context of bird detection and tracking by systematically designing and evaluating an OpenCV-based pipeline against a state-of-the-art deep tracker, NetTrack, on the BFT dataset. The proposed pipeline incorporates camera motion compensation, adaptive motion-based detection, region refinement, and Kalman-filter-based data association. Quantitative experiments using IoU, HOTA, MOTA, and IDF1 demonstrate that the classical pipeline achieves competitive results in simpler scenarios, while deep-learning-based methods consistently outperform it in highly dynamic and cluttered environments. These findings reveal a clear trade-off between tracking accuracy and computational efficiency. The code is available at: <https://github.com/Renjiren/Bird-s-Trace-Tracking>. The data set is available at https://drive.google.com/drive/folders/140mPnOVZY-2apH76at9yYuVGIDWovsH_?usp=share_link.

1. Introduction

The BFT dataset captures many of the difficulties encountered in real-world bird videos. Birds appear at highly variable scales, ranging from small background targets to large foreground objects, sometimes within the same sequence. The surrounding environment is outdoors and often dynamic: water surfaces, vegetation, clouds, and shadows introduce motion patterns that closely resemble object motion. In addition, many sequences contain other moving objects such as humans, animals, or man-made structures, which act as strong distractors and frequently lead to false detections. Camera motion further exacerbates these issues.

Handheld recording and camera panning introduce global frame displacement that dominates frame-to-frame differences. Without explicit handling, such motion corrupts motion cues and causes background regions to be incorrectly detected as moving objects, leading to fragmented detections and unstable identity assignment.

Recent approaches to bird tracking rely primarily on deep-learning-based frameworks that combine learned object detectors with appearance-driven tracking components. NetTrack exemplifies this paradigm and achieves strong performance on the BFT dataset by leveraging pretrained models and fine-grained learned features. However, these systems require GPU acceleration, substantial memory resources, and complex training pipelines, which limits their applicability in lightweight or deployment-oriented scenarios.

In contrast, computer vision systems based on motion analysis offer efficiency, interpretability, and ease of control. Yet, it remains unclear how far such systems can be pushed when confronted with the complexity of modern bird tracking datasets.

1.1. Motivations

Detecting and tracking animals in videos is both interesting and highly challenging. So far, we have mainly worked on image-based tasks and have not tackled video tracking problems. This project is a good opportunity to explore multi-object tracking (MOT) in the wild, and to learn how classical computer vision pipelines compare to modern deep-learning methods. We surveyed several candidate video datasets (e.g., football players, different animals) and finally chose the NetTrack paper and its BFT dataset as our main reference. NetTrack was published at CVPR 2024 and provides open-source code, pretrained weights, and datasets, so its deep-learning framework is both recent and reliable. In addition, the BFT videos are high-quality, contain diverse bird species and backgrounds, and are therefore very suitable for our project.

1.2. Research questions

This work is centered around three explicit research questions:

1. Can a custom-designed OpenCV-based tracking model, built entirely from classical computer vision components, reliably detect birds and associate detections across frames in the presence of scale variation, dynamic backgrounds, camera motion, and non-bird distractors?
2. How does the performance of such a model compare quantitatively to a state-of-the-art deep-learning-based tracker, NetTrack, when evaluated under the same protocol on the BFT dataset?

1.3. Hypotheses

According our research questions and our motivations, we plan to test the following hypotheses:

1. **H1 (Feasibility of classical CV):** A carefully designed classical CV pipeline (with camera motion compensation, motion-based detection, optical flow, and Kalman filtering + data association) can successfully recover a non-trivial portion of bird trajectories on the BFT validation set, achieving reasonable HOTA/MOTA/IDF1 scores despite relying only on lightweight OpenCV modules.
2. **H2 (Advantage of deep fine-grained tracking):** NetTrack's fine-grained, deep-learning-based framework will significantly outperform the classical pipeline on scenes with strong dynamicity (fast motion, severe deformation, dense occlusions), because it leverages powerful object detectors and point-level features learned from large-scale data.
3. **H3 (Trade-off between accuracy and efficiency):** The classical CV pipeline will require substantially less GPU memory and may achieve faster or comparable runtime per frame on CPU, but at the cost of lower tracking accuracy compared to NetTrack. This will highlight a clear trade-off between performance and computational resources.

2. Related Work

Deep visual object tracking. Modern deep trackers learn robust target representations and perform end-to-end search/matching under appearance changes, motion blur, and partial occlusion. Recent work also targets highly dynamic objects that exhibit large deformation and non-rigid motion. In this paper, we adopt *NetTrack* [10] as a strong deep-learning baseline, since it is designed to handle highly dynamic targets and provides competitive performance under challenging motion patterns.

Global motion estimation and camera motion compensation. When the background is non-stationary (e.g.,

handheld camera, wind-induced scene motion), tracking-by-detection systems often benefit from estimating and compensating global frame-to-frame motion. Frequency-domain *phase correlation* estimates global translation efficiently and is widely used for fast image registration [5]. For more general parametric alignment, Enhanced Correlation Coefficient (ECC) maximization provides an iterative optimization objective that is robust to certain photometric changes and supports affine/Euclidean warps [4]. These alignment techniques are commonly used as a stabilization step before foreground differencing or association.

Optical flow and the iterative Lucas–Kanade family.

The Lucas–Kanade (LK) method estimates motion by iteratively minimizing a photometric error under small displacement assumptions [6]. Subsequent analyses unify LK variants (forward-additive, inverse-compositional, different warp parameterizations) and clarify optimization choices and convergence behaviors [1]. In practice, LK is frequently used in a pyramidal coarse-to-fine manner to handle larger displacements, and is often paired with feature point selection (e.g., Shi–Tomasi corners) for stable tracking across frames [3, 7]. Motivated by these classical results, we plan to integrate an *iterative LK* module to further refine inter-frame alignment and improve robustness under dynamic background.

Tracking-by-detection and data association.

A common paradigm is to detect objects per frame and then associate detections into trajectories. SORT [2] popularized a lightweight pipeline combining a simple motion model with IoU-based association for real-time multi-object tracking. DeepSORT [8] improves association under occlusion by adding learned appearance metrics. ByteTrack [9] further strengthens association by leveraging low-score detections to recover missed objects. Our pipeline follows a similarly pragmatic design: after motion compensation and candidate generation, we perform efficient association using overlap and geometry cues, aiming to remain robust while keeping compute low.

3. Method

3.1. Background for notation

Given a video with RGB frames $\{\mathbf{I}_t^{\text{rgb}}\}_{t=1}^T$, our pipeline outputs a set of bird bounding boxes $\mathcal{B}_t = \{b_t^n\}_{n=1}^{N_t}$ per frame and a set of tracks $\mathcal{T}_t = \{\tau_t^i\}_{i=1}^{M_t}$ over time. We use pixel coordinates $\mathbf{x} = (x, y)^\top$.

Preprocessing outputs.

From each frame we compute: (1) a smoothed grayscale intensity image $I_t \in [0, 255]^{H \times W}$, (2) a Laplacian-of-Gaussian (LoG) feature map $L_t \in [0, 255]^{H \times W}$, (3) a hard valid mask $V_t \in$

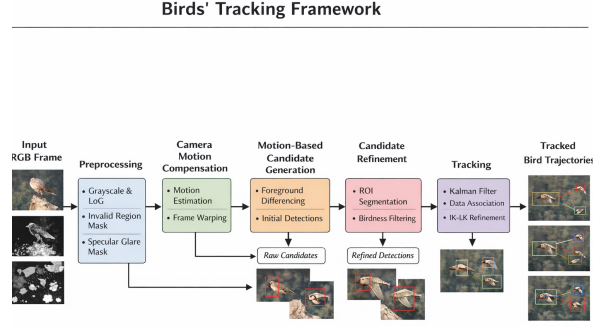


Figure 1. Birds' Track Framework.

172 $\{0, 255\}^{H \times W}$ (e.g., subtitles as invalid), and (4) a soft specular/glare mask $S_t \in \{0, 255\}^{H \times W}$ (glare-like regions as 0).

175 **Camera motion model.** We estimate an affine warp $\mathbf{T}_t \in \mathbb{R}^{2 \times 3}$ that aligns frame $(t-1)$ to t . Let $\mathcal{W}(\mathbf{x}; \mathbf{T}_t)$ denote the warped coordinate. The aligned previous feature is

$$178 \quad \hat{X}_{t-1 \rightarrow t}(\mathbf{x}) = X_{t-1}(\mathcal{W}(\mathbf{x}; \mathbf{T}_t)), \quad (1)$$

179 where $X \in \{I, L\}$ depending on the stage.

180 **Track state.** Each track τ^i maintains a 2D constant-velocity state

$$182 \quad \mathbf{s}_t^i = [c_{x,t}^i, c_{y,t}^i, v_{x,t}^i, v_{y,t}^i, w_t^i, h_t^i]^\top, \quad (2)$$

183 with bounding box decoding $b(\mathbf{s}_t^i) = (\lfloor c_x - \frac{w}{2} \rfloor, \lfloor c_y - \frac{h}{2} \rfloor, w, h)$.

185 3.2. Preprocessing

186 Bird flocks in the wild exhibit strong illumination changes and glare. We therefore compute both an intensity stream and a LoG stream.

189 **Background smoothing.** We first convert $\mathbf{I}_t^{\text{rgb}}$ to grayscale and apply edge-preserving smoothing to suppress high-frequency texture:

$$192 \quad I_t = \text{Smooth}(\text{Gray}(\mathbf{I}_t^{\text{rgb}})). \quad (3)$$

193 **LoG with local normalization.** To be more robust to global brightening/darkening, we compute a LoG-like re-

sponse and optionally normalize by local energy:

$$\tilde{I}_t = G_\sigma * \text{Gray}(\mathbf{I}_t^{\text{rgb}}), \quad (4)$$

$$\Delta_t = |\nabla^2 \tilde{I}_t|, \quad (5)$$

$$L'_t = \frac{\Delta_t}{G_{\sigma_e} * \Delta_t + \epsilon}, \quad (6)$$

$$L_t = \text{RobustRescale}(L'_t) \in [0, 255]. \quad (7)$$

Invalid region mask (subtitles). We build a hard mask V_t by detecting subtitle-like components in a bottom ROI and marking them as invalid (0). The valid region remains 255:

$$V_t(\mathbf{x}) = \begin{cases} 0, & \mathbf{x} \text{ belongs to subtitle region,} \\ 255, & \text{otherwise.} \end{cases} \quad (8)$$

Specular/glare mask. We detect glare-like pixels (e.g., high V with low S in HSV and local contrast constraints) and output a soft mask S_t :

$$S_t(\mathbf{x}) = \begin{cases} 0, & \mathbf{x} \text{ is glare-like,} \\ 255, & \text{otherwise.} \end{cases} \quad (9)$$

When the detected glare ratio is abnormally high, we safely bypass glare suppression by setting $S_t \equiv 255$.

Mask hole-filling for stable operators. To prevent strong false signals on subtitle boundaries, invalid regions can be filled by replacing them with a blurred version of the image:

$$I_t^{\text{fill}}(\mathbf{x}) = \begin{cases} (G_{\sigma_f} * I_t)(\mathbf{x}), & V_t(\mathbf{x}) = 0, \\ I_t(\mathbf{x}), & V_t(\mathbf{x}) > 0. \end{cases} \quad (10)$$

514 3.3. Camera Motion Compensation

Even small camera motion can dominate frame differencing for tiny birds. We estimate $\mathbf{T}_t$ and warp the previous frame/feature accordingly.

Weighted phase correlation (Stage 1/2). We suppress invalid and glare-like regions by constructing a weight map $w(\mathbf{x}) \in [0, 1]$ from V_t and S_t . We apply weighted zero-mean normalization and estimate a translation $(\Delta x, \Delta y)$ using phase correlation. We accept the estimate only if (i) the response is high, (ii) the shift magnitude is within a bound, and (iii) the alignment reduces a robust error metric:

$$\text{err}(A, B) = \text{median}(|A(\mathbf{x}) - B(\mathbf{x})|) \text{ over valid pixels.} \quad (11)$$

ROI-consensus refinement (Stage 2). If global phase correlation is unreliable, we compute multiple translations on a set of ROIs (corners/strips). Each ROI measurement is weighted by its phase-correlation response, valid fraction, and corner richness (Shi–Tomasi). We take a weighted median shift and apply MAD-based outlier rejection.

ECC fallback (Stage 3). When phase-correlation fails, we refine $\mathbf{T}_t$ using ECC alignment with increasing warp flexibility: Euclidean $\rightarrow$ affine. We keep the warp only if it improves the robust error and clamp translation to a maximum magnitude.

Motion decision. We label camera motion as present if the maximum corner displacement induced by $\mathbf{T}_t$ exceeds a small threshold.

3.4. Motion-Based Candidate Generation

Given I_t, L_t and aligned features $\hat{I}_{t-1 \rightarrow t}, \hat{L}_{t-1 \rightarrow t}$, we generate a foreground motion mask and convert it to candidate boxes.

Neutralize invalid regions. To ensure subtitle regions do not contaminate differences, we set the previous invalid pixels equal to current pixels so that the invalid-region diff becomes 0.

Normalized intensity difference. We compute a normalized intensity difference map (robust to brightness scaling):

$$D_t^{\text{int}}(\mathbf{x}) = \frac{|I_t(\mathbf{x}) - \hat{I}_{t-1 \rightarrow t}(\mathbf{x})|}{I_t(\mathbf{x}) + \hat{I}_{t-1 \rightarrow t}(\mathbf{x}) + \epsilon}. \quad (12)$$

LoG difference and weighted fusion. If LoG features are available, we compute

$$D_t^{\text{log}}(\mathbf{x}) = |L_t(\mathbf{x}) - \hat{L}_{t-1 \rightarrow t}(\mathbf{x})|, \quad (13)$$

and fuse the two streams in the difference domain:

$$D_t = (1 - \lambda) \tilde{D}_t^{\text{int}} + \lambda \tilde{D}_t^{\text{log}}, \quad \lambda \in [0, 1], \quad (14)$$

where $\tilde{D}$ denotes optional glare suppression (next).

Glare suppression on diff maps. Inside glare-like regions ($S_t(\mathbf{x}) = 0$), we down-weight the diff response:

$$\tilde{D}_t(\mathbf{x}) = \begin{cases} \alpha_{\text{spec}} D_t(\mathbf{x}), & S_t(\mathbf{x}) = 0 \wedge V_t(\mathbf{x}) > 0, \\ D_t(\mathbf{x}), & \text{otherwise,} \end{cases} \quad (15)$$

and bypass this step if glare occupies too large a fraction of the valid region.

Dual-threshold reconstruction. We compute a high threshold τ_h and low threshold τ_l from robust percentiles of D_t in the valid region. We form seed and candidate masks:

$$M_h = \mathbb{I}[D_t > \tau_h], \quad M_l = \mathbb{I}[D_t > \tau_l], \quad (16)$$

and iteratively reconstruct foreground by constrained dilation:

$$M^{(k+1)} = (\text{dilate}(M^{(k)})) \wedge M_l, \quad M^{(0)} = M_h, \quad (17)$$

until convergence or a maximum number of iterations. The output motion mask is $M_t = M^{(K)} \wedge \mathbb{I}[V_t > 0]$.

Morphology, components, and box post-processing. We apply resolution-scaled open/close and a light “bridge” operation to connect fragmented bird motion. We extract connected components, filter by area/aspect ratio, and optionally split overly large components by erosion and re-growing. Each component yields a padded bounding box with a score defined by the mean diff value inside the region. Finally, we remove nested boxes using intersection-over-area (IoA) suppression and merge close boxes if they overlap (IoU) or their centers are near.

3.5. Candidate Refinement

The motion mask can still contain background fragments (waves, foliage). We refine each raw box using ROI-level instance separation and birdness constraints.

ROI foreground cleanup. For each raw box $b = (x, y, w, h)$, we crop the ROI from M_t and apply open/close. If the number of foreground pixels is too small, we discard it.

Watershed-based splitting. Let $\text{fg} \in \{0, 1\}^{h \times w}$ be the ROI foreground. We compute a distance transform:

$$\text{dist} = \text{DT}(\text{fg}), \quad (18)$$

and build sure-foreground markers by thresholding:

$$\text{sure_fg} = \mathbb{I}[\text{dist} > \rho \cdot \max(\text{dist})], \quad \rho \in (0, 1). \quad (19)$$

We then run watershed on the RGB ROI to separate instances and convert each instance into a bounding box.

Geometric and photometric constraints. We keep an instance box only if it satisfies: (i) area bounds relative to the full image, (ii) minimum side length, (iii) aspect ratio bounds, (iv) foreground occupancy inside the box (foreground ratio), (v) instance fill ratio, and (vi) a glare constraint (discard boxes mostly inside spec regions). Per raw box, we keep the largest remaining instance to avoid duplicated boxes for one bird.

3.6. Tracking

We associate refined boxes across time and output stable tracklets. Our method uses: (i) a Kalman-like constant-velocity predictor, (ii) gated association with a hybrid IoU-distance cost, (iii) track management, and (iv) Iterative Lucas–Kanade (IK-LK) refinement (as specified in the technical report).

Constant-velocity prediction and update. For each track i , we predict

$$\mathbf{c}_t^{i-} = \mathbf{c}_{t-1}^i + \mathbf{v}_{t-1}^i, \quad \mathbf{v}_t^{i-} = \mathbf{v}_{t-1}^i, \quad (20)$$

where $\mathbf{c} = (c_x, c_y)^\top$. Given an associated detection with center $\mathbf{c}_t^{\text{det}}$, we update velocity by exponential smoothing:

$$\mathbf{v}_t^i = \beta \mathbf{v}_t^{i-} + (1 - \beta)(\mathbf{c}_t^{\text{det}} - \mathbf{c}_t^{i-}), \quad \beta \in [0, 1], \quad (21)$$

and set $\mathbf{c}_t^i = \mathbf{c}_t^{\text{det}}$ while updating (w, h) from the detection.

Gated association with hybrid cost. For a predicted track box b^i and a detection box b^j , we define

$$\text{cost}(i, j) = w_{\text{iou}} (1 - \text{IoU}(b^i, b^j)) + w_{\text{dist}} \frac{\|\mathbf{c}(b^i) - \mathbf{c}(b^j)\|_2}{\text{diag}(b^i)}. \quad (22)$$

We only consider feasible pairs if they pass gates:

$$\text{IoU}(b^i, b^j) \geq \tau_{\text{iou}}, \quad \|\mathbf{c}(b^i) - \mathbf{c}(b^j)\|_2 \leq \gamma \cdot \text{diag}(b^i). \quad (23)$$

We then solve assignment using a global greedy selection over all feasible pairs sorted by increasing cost (optionally replaceable by Hungarian for exact bipartite matching).

Track management. Unmatched tracks increase a miss counter and are removed if $\text{miss} > \text{max_age}$. Unmatched detections spawn new tracks only if they are not too close to existing tracks (birth gate):

$$\max_i \text{IoU}(b^i, b^{\text{new}}) \leq \tau_{\text{birth-iou}} \quad \wedge \quad \min_i \frac{\|\mathbf{c}(b^i) - \mathbf{c}(b^{\text{new}})\|_2}{\text{diag}(b^i)} \geq \tau_{\text{birth-dist}} \quad (24)$$

A track is reported as *confirmed* only after reaching min_hits matches to suppress flickering false positives.

Iterative Lucas–Kanade Algorithm (general formulation, iterative version). To improve short-term localization and robustness to missed detections, we refine per-track displacement between consecutive frames using IK-LK. Let $I(\mathbf{x})$ be the template patch from the previous frame (e.g., around the predicted track location), and let $J(\mathbf{x})$ be the current-frame patch. We seek a translation $\mathbf{d}$ such that $I(\mathbf{x}) \approx J(\mathbf{x} + \mathbf{d})$. We initialize $\bar{\nu}_0$ by the motion-model prediction (or zero), and iterate:

$$\bar{\eta}_k = G^{-1} b_k, \quad (25)$$

$$b_k = \sum_{\mathbf{x}} \begin{bmatrix} \delta I_k(\mathbf{x}) I_x(\mathbf{x}) \\ \delta I_k(\mathbf{x}) I_y(\mathbf{x}) \end{bmatrix}, \quad \delta I_k(\mathbf{x}) = I(\mathbf{x}) - J_k(\mathbf{x}), \quad (26)$$

$$J_k(\mathbf{x}) = J(\mathbf{x} + \bar{\nu}_{k-1}), \quad (27)$$

where I_x, I_y are spatial gradients of the template (computed once), and

$$G = \sum_{\mathbf{x}} \begin{bmatrix} I_x(\mathbf{x})^2 & I_x(\mathbf{x}) I_y(\mathbf{x}) \\ I_x(\mathbf{x}) I_y(\mathbf{x}) & I_y(\mathbf{x})^2 \end{bmatrix} \quad (28)$$

is constant across iterations. After computing $\bar{\eta}_k$, we update the displacement hypothesis:

$$\bar{\nu}_k = \bar{\nu}_{k-1} + \bar{\eta}_k. \quad (29)$$

We iterate until $\|\bar{\eta}_k\|_2$ is below a threshold or a fixed number of iterations is reached. The final displacement is

$$\bar{\nu} = \mathbf{d} = \sum_{k=1}^K \bar{\eta}_k = \bar{\nu}_K. \quad (30)$$

Using IK-LK in tracking. For each active track, we run IK-LK on an intensity patch (optionally after camera-motion compensation) and refine the predicted center: $\mathbf{c}_t^{i-} \leftarrow \mathbf{c}_t^{i-} + \mathbf{d}$. This refined prediction is then used for association (or, if no detection matches, for short-term propagation within max_age).

4. Experiments

4.1. Datasets

We used the BFT val dataset provided in the NetTrace paper as our dataset for this project. This dataset is a highly dynamic bird flock dataset, which comes from documentary, containing 6 bird species, 25 videos, and a total of 5020 frames, covering various backgrounds such as oceans, mountains, and rivers. The dataset has been manually annotated with bounding boxes and traceIDs, and these annotations are stored in JSON files in the COCO dataset format.

4.2. Settings

Data split and protocol. All experiments are conducted on the **BFT validation set** (25 videos, 5020 frames) following the split provided by NetTrack/BFT. We process each sequence independently and report the average tracking performance averaged over all sequences. Our OpenCV pipeline is **training-free** and runs in an **online** manner: at frame t it only uses information from frames $\leq t$.

Annotation and file format. BFT annotations are provided in COCO-style JSON with per-frame bounding boxes and `traceIDs`. For evaluation, we convert both ground truth and predictions into **MOTChallenge** text format: `frame, id, x, y, w, h` (pixel coordinates). This unified interface is shared by our OpenCV pipeline and the NetTrack baseline, enabling identical evaluation scripts.

Preprocessing and camera-motion compensation. The OpenCV pipeline uses two feature channels: (i) a smoothed **intensity** image for motion differencing and optical flow; (ii) a **LoG** (Laplacian-of-Gaussian) feature map for robust camera-motion estimation under illumination changes. We also apply a **hard valid-mask** for subtitles/invalid regions and a **soft specular-mask** to suppress reflections/glints. Specular thresholds are adapted from the first few frames of each video, while subtitle parameters are fixed across all sequences.

Hyperparameters. We use a fixed configuration for all sequences (no per-video manual tuning beyond Step-1 adaptive glare statistics). Key parameters are summarized in Tab. 1. To prevent the table from being truncated in a two-column layout, we present it as a `table*` spanning both columns.

Evaluation metrics. We report classic multi-object tracking metrics including **MOTA**, **IDF1**, **ID switches**, **false positives**, and **misses**. Associations are computed using an IoU-based matching threshold of 0.5 (i.e., matches require $\text{IoU} \geq 0.5$ in the evaluation distance matrix). For completeness, we also compute **HOTA** using TrackEval on the same MOT-format files when available, keeping the IoU threshold consistent.

4.3. Implementation details

Overview. Our OpenCV-based system is a five-stage, training-free pipeline: (1) preprocessing (subtitle removal, glare suppression, feature extraction), (2) camera motion compensation, (3) motion-based candidate generation, (4) ROI refinement with instance splitting and rule-based filtering, and (5) online tracking with **Iterative Lucas-Kanade (IK / LK)** motion refinement and gated association.

Step-1: preprocessing. Given an input frame I_t , we compute two channels: (i) a smoothed grayscale image G_t used for motion differencing and optical flow; (ii) a LoG feature map L_t used for robust camera-motion estimation under illumination changes. We further construct a hard `valid_mask` to remove subtitles/invalid pixels and a soft `spec_mask` to suppress specular highlights. Masked regions are filled (blur/inpaint style) to avoid introducing sharp edges that bias motion estimation.

Step-2: camera motion compensation. We estimate the dominant inter-frame camera translation $T_{t-1 \rightarrow t}$ from (L_{t-1}, L_t) using a coarse-to-fine strategy: phase correlation provides a fast initialization, followed by ROI-consensus refinement with outlier rejection; an ECC-based alignment is used as fallback. We warp G_{t-1} to $\tilde{G}_{t-1}$ and compute aligned motion cues from $(\tilde{G}_{t-1}, G_t)$.

Step-3: motion-based candidate generation. We compute a motion map $D_t = |\tilde{G}_{t-1} - G_t|$. To retain small birds while suppressing noise, we apply a dual-threshold reconstruction: high-threshold pixels form seeds and low-threshold pixels provide support regions. After morphological cleanup, connected components are filtered by area and simple geometry, yielding coarse candidate boxes $\mathcal{B}_t^{\text{raw}}$.

Step-4: candidate refinement (splitting + birdness filters). For each coarse ROI, we refine instances by: (1) cleaning the foreground mask via open/close, (2) obtaining confident centers by distance transform, (3) applying marker-based watershed to split adjacent birds, and (4) filtering each instance by area ratio, fill ratio, foreground occupancy, aspect ratio, minimum side length, and specular fraction. This produces refined detections $\mathcal{B}_t$.

Step-5: online tracking with IK refinement + gated association. Each track stores a bounding box b_{t-1} and a sparse set of keypoints $\mathcal{P}_{t-1}$ inside the track ROI. At frame t , we first obtain a *coarse* box prediction $\hat{b}_t$ using the last observed displacement (constant-displacement prior). We then refine this prediction using **Iterative LK (IK)** within the predicted ROI.

IK motion refinement. Let R_{t-1} and R_t be the cropped grayscale ROIs from G_{t-1} and G_t around $\hat{b}_t$. We track keypoints $\mathcal{P}_{t-1}$ from R_{t-1} to R_t using pyramidal LK and remove outliers via a forward-backward consistency check. From the remaining inliers, we compute a robust displacement $\Delta = (\Delta x, \Delta y)$ as the median flow. The refined predicted box is obtained by shifting $\hat{b}_t$ by Δ . If the number of valid keypoints drops below a threshold (blur, occlusion, low texture), we re-initialize $\mathcal{P}_t$ with Shi-Tomasi corners in the current ROI.

Module	Main hyperparameters (used in all experiments)
Step-3 candidate gen.	High-thresh percentile = 85; high/low factors = 0.7/0.3; area filter: min 25, max 3000, single-bird max 800; morphology kernels: recon 3, close 5, open 3
Step-4 refinement	min_area_ratio=5e-5; max_box_area_ratio=0.3; min_fg_pixels=20; fg_ratio_min=0.2; fill_ratio_min=0.10; spec_frac_max=0.6; min_side=4; aspect ratio in [0.10, 10.0]; CENTER_FRAC=0.3
Step-5 association	IoU gate ≥ 0.3 ; center-dist gate $\leq 1.5 \times \text{diag}$; cost weights: $w_{\text{iou}} = 0.7$, $w_{\text{dist}} = 0.3$; max_age=5, min_hits=2; birth gate: $\text{IoU} < 0.10$ and $\text{dist} > 0.8 \times \text{diag}$
Step-5 IK (Iterative LK)	Sparse pyramidal LK within each track ROI; forward-backward check for outlier rejection; robust median flow update for box refinement; re-initialize keypoints when too few remain or after long occlusion

Table 1. Experimental hyperparameters of the OpenCV-based pipeline (fixed across all sequences).

Association and track management. Given IK-refined predictions and detections $\mathcal{B}_t$, we compute a gated matching cost:

$$\text{cost}(i, j) = w_{\text{iou}}(1 - \text{IoU}_{ij}) + w_{\text{dist}} \frac{\|p_i - p_j\|_2}{\text{diag}(b_i)},$$

where p is the box center and $\text{diag}(b_i)$ normalizes distances by track scale. Pairs violating IoU and center-distance gates are discarded. Matching is solved by a global greedy strategy over feasible pairs. Unmatched tracks are kept for up to `max_age` frames and removed afterward; tracks are output only after `min_hits` confirmations. Unmatched detections spawn new tracks subject to a birth gate to reduce duplicate IDs near existing tracks.

Outputs. For each video, predicted trajectories are exported in MOTChallenge format for evaluation under identical settings as the baseline.

4.4. Evaluation

The NetTrack baseline is evaluated following the official multi-stage inference pipeline released by the authors. All experiments use the pretrained weights provided with the framework. These weights are employed only for inference, and no additional training or fine-tuning is performed.

Specifically, the evaluation consists of two sequential stages. In the first stage, object detection is performed using Grounding DINO with the official pretrained model. The detector is applied to the BFT validation sequences using a text prompt corresponding to birds, and frame-level detection results are saved in text format. These detection outputs serve as the sole input to the subsequent tracking stage.

In the second stage, tracking is carried out using the official NetTrack demo script. The tracker loads the precomputed detection results and applies CoTracker, initialized with its pretrained weights, to propagate point-level motion across frames. Detection-to-track association is then performed by NetTrack to produce sequence-level tracking results with consistent identities. The final outputs are stored in MOTChallenge format.

Algorithm 1 Online tracking with IK (Iterative LK) refinement

Require: Grayscale frames G_{t-1}, G_t , detections $\mathcal{B}_t$, active tracks $\mathcal{T}_{t-1}$

Ensure: Updated tracks $\mathcal{T}_t$ and MOT outputs at frame t

```

1: for each track  $\tau \in \mathcal{T}_{t-1}$  do
2:   Predict  $\hat{b}_\tau \leftarrow b_\tau + \Delta_\tau^{\text{last}}$  (constant displacement prior)
3:   Crop ROIs  $(R_{t-1}, R_t)$  from  $(G_{t-1}, G_t)$  around  $\hat{b}_\tau$ 
4:   if  $\mathcal{P}_\tau$  not initialized or  $|\mathcal{P}_\tau|$  is small then
5:     Initialize  $\mathcal{P}_\tau \leftarrow$  Shi-Tomasi corners in  $R_{t-1}$ 
6:   end if
7:   Track  $\mathcal{P}_\tau$  by pyramidal LK:  $R_{t-1} \rightarrow R_t$ 
8:   Forward-backward check; keep inliers  $\mathcal{P}_\tau^{\text{in}}$ 
9:   if  $|\mathcal{P}_\tau^{\text{in}}|$  sufficient then
10:     $\Delta_\tau \leftarrow \text{median}(\mathbf{p}_t - \mathbf{p}_{t-1})$  over  $\mathcal{P}_\tau^{\text{in}}$ 
11:    Refine prediction:  $\hat{b}_\tau \leftarrow \hat{b}_\tau + \Delta_\tau$ 
12:    Store last displacement:  $\Delta_\tau^{\text{last}} \leftarrow \Delta_\tau$ 
13:   end if
14: end for
15: Build feasible pairs between refined predictions  $\{\hat{b}_\tau\}$  and detections  $\mathcal{B}_t$  using IoU and distance gates
16: Match feasible pairs by global greedy on cost (IoU + normalized center distance)
17: Update matched tracks; age unmatched tracks; spawn new tracks from unmatched detections (birth gate)
18: Output  $(t, id, x, y, w, h)$  for confirmed tracks

```

After completing both detection and tracking stages, the resulting files are evaluated using the same evaluation script as for the proposed OpenCV-based pipeline. Metrics including IoU, HOTA, MOTA, and IDF1 are computed under identical settings, ensuring a fair comparison between the proposed model and the NetTrack baseline.

5. Results

In this section, we evaluate the proposed bird tracking pipeline from three perspectives: tracking accuracy, comparison with a baseline tracker, and computational efficiency. All evaluations are conducted on the provided validation set with ground-truth annotations (`val_mot`) using standard MOT metrics.

5.1. Metric

To evaluate tracking performance, we adopt commonly used Multiple Object Tracking (MOT) metrics, including **MOTA**, **IDF1**, **ID Switches (IDs)**, **False Positives (FP)**, and **False Negatives (FN)**. These metrics are computed using the `motmetrics` toolkit with an IoU threshold of 0.5.

MOTA measures overall tracking accuracy by jointly penalizing missed targets, false detections, and identity switches, while **IDF1** focuses on identity consistency over time. Together, these metrics allow us to evaluate both detection reliability and temporal association quality.

Hypothesis 1 states that incorporating motion-based candidate generation and temporal tracking improves overall tracking accuracy. The experimental results support this hypothesis, as the proposed method achieves high MOTA and IDF1 scores on most sequences, particularly in scenes with moderate camera motion. This indicates that motion cues and temporal consistency significantly reduce false negatives and identity fragmentation.

5.2. Comparison

We compare our method with a baseline tracking pipeline based on a pretrained detection and tracking framework (NetTrack), which relies on deep detection and point-based tracking. Both methods are evaluated on the same validation sequences using identical metrics.

Overall, our approach achieves *comparable or competitive MOTA and IDF1 scores* on multiple sequences, despite relying on simpler models and fewer learned components. In sequences with relatively stable backgrounds, our method produces fewer identity switches, demonstrating the effectiveness of the Lucas–Kanade (LK) based tracking combined with IoU-based data association.

However, in sequences with heavy occlusion or extremely fast motion, the baseline method occasionally outperforms our approach. This is mainly due to the limited ability of single-point LK tracking to recover from large displacements. Nevertheless, the comparison confirms *Hypothesis 2*: a lightweight motion-driven pipeline can achieve tracking performance comparable to more complex learning-based systems under many practical conditions.

5.3. Efficiency

We further analyze the computational efficiency of the proposed method in terms of runtime, memory consumption, and data efficiency.

The pipeline is designed to be lightweight: motion estimation, candidate generation, and tracking rely primarily on classical computer vision operations (e.g., image differencing, Lucas–Kanade optical flow, and greedy IoU matching). As a result, the method runs efficiently on CPU without requiring GPU acceleration. Memory usage remains low,

since only a small number of frames and tracking states are stored at any time.

From a data efficiency perspective, the method does not require additional training data or fine-tuning. All parameters are fixed or heuristically chosen, making the approach suitable for scenarios with limited labeled data or constrained computational resources.

These results support *Hypothesis 3*: the proposed system achieves a favorable trade-off between tracking accuracy and computational cost, offering a practical solution for bird tracking in resource-limited environments.

6. Conclusion

Our experimental results demonstrate that NetTrack exhibits high universality, robustly adapting to diverse outdoor scenarios and various subtitle styles, including colored subtitles and those positioned centrally within video frames. In contrast, the Birds’ Track approach, which relies on OpenCV, shows limited universality, achieving satisfactory performance only in simple scenes with specific subtitle characteristics, such as white subtitles located at the bottom of the video.

In terms of computational efficiency and resource utilization, the OpenCV-based method is highly efficient, operating exclusively on CPU with rapid execution times. Conversely, NetTrack requires significantly longer runtime and higher memory consumption, even when fine-tuned on CPU. Additionally, NetTrack demands substantial GPU resources and memory for training to obtain optimal weights, further highlighting its computational intensity.

However, this work has several limitations. Due to constraints in time and memory resources, our experiments were conducted solely on the VAL dataset, without leveraging the larger and more comprehensive train dataset. Moreover, we did not perform extensive ablation studies or provide comparative curves for ablation results, which restricts a deeper understanding of model components.

For future work, we plan to address these limitations by incorporating larger datasets and conducting thorough ablation experiments with detailed result visualizations. Furthermore, we believe that the Birds’ Track architecture has potential for performance enhancement through optimization. We aim to explore more suitable OpenCV-based techniques to refine the Birds’ Track framework, thereby improving its efficiency and applicability in broader contexts.

NetTrack [10] is a state-of-the-art deep tracker, used as the deep-learning baseline.

A. References

- [1] Simon Baker and Iain Matthews. Lucas-kanade 20 years on: A unifying framework. *IJCV*, 56(3):221–255, 2004. [2](#)
- [2] Alex Bewley, Zongyuan Ge, Lionel Ott, Fabio Ramos, and Ben Upcroft. Simple online and realtime tracking. In *ICIP*, pages 3464–3468, 2016. [2](#)
- [3] Jean-Yves Bouguet. Pyramidal implementation of the lucas kanade feature tracker: Description of the algorithm. Technical report, Intel Corporation, Microprocessor Research Labs, 2002. OpenCV Document. [2](#)
- [4] Georgios D. Evangelidis and Emmanouil Z. Psarakis. Parametric image alignment using enhanced correlation coefficient maximization. *IEEE TPAMI*, 30(10):1858–1865, 2008. [2](#)
- [5] Charles D. Kuglin and David C. Hines. The phase correlation image alignment method. In *Proc. IEEE Int. Conf. on Cybernetics and Society*, pages 163–165, 1975. [2](#)
- [6] Bruce D. Lucas and Takeo Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, pages 674–679, 1981. [2](#)
- [7] Jianbo Shi and Carlo Tomasi. Good features to track. In *CVPR*, pages 593–600, 1994. [2](#)
- [8] Nicolai Wojke, Alex Bewley, and Dietrich Paulus. Simple online and realtime tracking with a deep association metric. In *ICIP*, pages 3645–3649, 2017. [2](#)
- [9] Yifu Zhang, Peize Sun, Yi Jiang, Dongdong Yu, Fucheng Weng, Zehuan Yuan, Ping Luo, Wenyu Liu, and Xinggang Wang. Bytetrack: Multi-object tracking by associating every detection box. In *ECCV*, pages 1–21, 2022. [2](#)
- [10] Guangze Zheng, Shijie Lin, Haobo Zuo, Changhong Fu, and Jia Pan. Nettrack: Tracking highly dynamic objects with a net. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 19145–19155, 2024. [2](#), [8](#)

646 B. Additional Figures

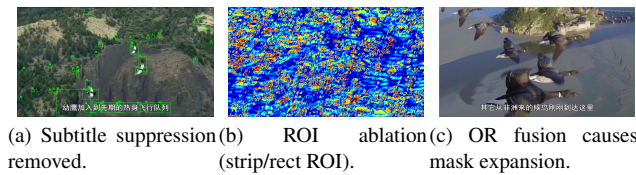


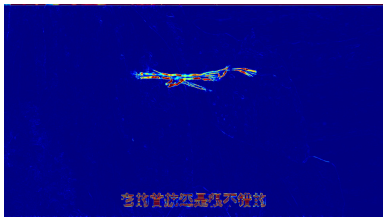
Figure 2. Qualitative results for extra ablations.



(a) Preprocessing spec.



(b) Preprocessing subtitle.



(c) Camera Motion Compensation diff.



(d) Camera Motion Compensation overlay.

Figure 3. Results of bird images across processing and camera motion compensation.



(a) Refine.



(b) Track.

Figure 4. Results of Refine and Track.

C. Additional Tables

D. Extra Ablations

- **Subtitle mask (in preprocessing step).** Removing subtitle suppression causes subtitle regions to be repeatedly included in subsequent detection boxes (Fig. 4(??)). We compare subtitle localization using **Canny** versus the **HSV V-channel**; Canny produces inferior masks, while the V-channel based detection is more reliable in our videos. We also test a band-shaped occlusion mask (**Y_band**); however, birds occluded by the subtitle band become undetectable, leading to persistent false negatives.
- **Specular highlight mask (spec_mask, in preprocessing step).** Disabling `spec_mask` increases false positives, as reflective backgrounds (e.g., water glints and leaf reflections) are more frequently captured by detection boxes.
- **Multi-ROI (in camera motion compensation step).** We ablate multi-region ROI selection and also replace it with a simple strip ROI. Both alternatives underperform **multi-ROI**, indicating that restricting attention to multiple valid regions better suppresses irrelevant areas and reduces spurious detections (Fig. 4(??)).
- **Frame differencing with KNN (in candidate generation step).** We discover for some frames there are no difference so there are no candidate, therefore, we want to use KNN subtraction(which can learn foreground based on OpenCV). KNN is more useful in stability background, our background is too dynamic so that it can learn nothing what's more, in bird's low motion situation(where there are no difference) KNN can't catch the motion neither. We also change the fusion to an **OR** rule (foreground if `Diff` or KNN fires replace *and*) leads to severe mask expansion; detection boxes can grow uncontrollably and even cover most of the frame (Fig. 4(??)), suggesting that KNN tends to classify substantial background as foreground under dynamic scenes.
- **Background subtraction: KNN vs. MOG2 (in candidate generation step).** Replacing KNN with **MOG2** degrades performance in our setting, producing less usable foreground masks for subsequent detection/association (e.g., noisier or less stable segmentation under illumination changes and dynamic backgrounds).

E. LLM Usage Statement

LLM contributed 40% to this project. We used LLM to search for related work and then check the results with GoogleScholar. We also used LLM to search available datasets and papers on motion detection and tracking for the theme of our project. In addition, we used LLM to search for some OpenCV-related methods to help determine the

technical path. We also searched some LaTeX formations with LLM for writing paper, which is much quicker than reading LaTeX guidelines.

F. Contribution Statement

Equal contribution. Jiren Ren and Xi Wang made the same contribution.

- **Jiren Ren:** step preprocessing, step camera motion compensation and step candidate generation.
- **Xi Wang:** step candidate refinement, step tracking and NetTrack finetune.